

Car Price Prediction Dashboard Documentation

Que – 2

Name – Rupam Kumari Jaiswal

Aim

To develop an interactive dashboard that predicts **used car prices** based on features like **year, fuel type, and mileage**, and displays results with charts.

Objectives

- Build a regression model to predict car prices.
- Visualize feature importance to understand key factors.
- Provide a user-friendly interface for input.
- Display predicted prices with charts for better interpretation.

Requirements

- **Software:** Python, Jupyter Notebook/VS Code, Streamlit/Flask.
- **Libraries:** Pandas, NumPy, Scikit-learn, Matplotlib/Seaborn, Streamlit.
- **Dataset:** Car details with columns like `year`, `fuel_type`, `mileage`, `price`.

Code –

```
import pandas as pd

import numpy as np

def load_and_clean_data():

    data = {

        'Year': [2014, 2013, 2017, 2011, 2014, 2015, 2016, 2015],

        'Fuel_Type': ['Petrol', 'Diesel', 'Petrol', 'Diesel', 'Diesel', 'Petrol',

'Diesel', 'Petrol'],

        'Driven_kms': [27000, 43000, 6900, 5200, 42450, 2071, 41000, 42000],

        'Selling_Price': [3.35, 4.75, 7.25, 2.85, 4.60, 9.25, 6.75, 6.50] # In

thousands/millions

    }

    df = pd.DataFrame(data)
```

```

df = df.dropna()

current_year = 2026

df['Age'] = current_year - df['Year']

df = df.drop(columns=['Year'])

return df

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

def train_model(df):

    X = df.drop(columns=['Selling_Price'])

    y = df['Selling_Price']

    categorical_features = ['Fuel_Type']

    numerical_features = ['Driven_kms', 'Age']

    preprocessor = ColumnTransformer(

        transformers=[

            ('num', 'passthrough', numerical_features),

            ('cat', OneHotEncoder(), categorical_features)

        ])

    model_pipeline = Pipeline(steps=[

```

```

        ('preprocessor', preprocessor),

        ('regressor', RandomForestRegressor(n_estimators=100, random_state=42))

    ])

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    model_pipeline.fit(X_train, y_train)

    return model_pipeline, X, preprocessor

import streamlit as st

import plotly.express as px

st.set_page_config(page_title="Car Price Predictor", layout="wide")

st.title("🚗 Used Car Price Prediction Dashboard")

st.write("Predict the valuation of a used car based on its historical features.")

df = load_and_clean_data()

model, X, preprocessor = train_model(df)

col1, col2 = st.columns([1, 2])

with col1:

    st.header("🔗 Car Specifications")

    input_year = st.slider("Manufacturing Year", min_value=2010, max_value=2026,
value=2018)

    input_age = 2026 - input_year

```

```
input_mileage = st.number_input("Driven Kilometers (Mileage)", min_value=0,
value=30000, step=1000)
```

```
fuel_options = df['Fuel_Type'].unique()
```

```
input_fuel = st.selectbox("Fuel Type", options=fuel_options)
```

```
predict_btn = st.button("Predict Selling Price", type="primary")
```

```
with col2:
```

```
if predict_btn:
```

```
    # Construct dataframe for prediction matching training format
```

```
    user_data = pd.DataFrame({
        'Fuel_Type': [input_fuel],
        'Driven_kms': [input_mileage],
        'Age': [input_age]
    })
```

```
    prediction = model.predict(user_data)[0]
```

```
    st.success(f"Predicted Value: ${prediction:.2f} Lakhs")
```

```
    st.subheader("📊 Feature Importance Analysis")
```

```
    importances = model.named_steps['regressor'].feature_importances_
```

```
    cat_encoder = model.named_steps['preprocessor'].named_transformers_['cat']
```

```
    encoded_cat_names = cat_encoder.get_feature_names_out(['Fuel_Type']).tolist()
```

```
    feature_names = ['Driven_kms', 'Age'] + encoded_cat_names
```

```

importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=True)

fig_importance = px.bar(
    importance_df,
    x='Importance',
    y='Feature',
    orientation='h',
    title="What factors influence the price most?",
    labels={'Importance': 'Relative Importance Factor'},
    color='Importance',
    color_continuous_scale='Blues'
)

st.plotly_chart(fig_importance, use_container_width=True)

st.subheader("📊 Mileage vs Price Trend")
fig_scatter = px.scatter(
    df, x='Driven_kms', y='Selling_Price', color='Fuel_Type',
    title="Dataset Distribution: Mileage vs Price",
    labels={'Driven_kms': 'Kilometers Driven', 'Selling_Price': 'Price'}
)

fig_scatter.add_scatter(x=[input_mileage], y=[prediction], mode='markers',
                        marker=dict(size=15, color='red', symbol='star'),
                        name='Your Prediction')

st.plotly_chart(fig_scatter, use_container_width=True)

```

```

else:

    st.info("Adjust the car specifications on the left and click **Predict Selling Price** to generate charts.")

```

screenshot –

```

1  import pandas as pd
2  import numpy as np
3
4  def load_and_clean_data():
5
6      data = {
7          'Year': [2014, 2013, 2017, 2011, 2014, 2015, 2016, 2015],
8          'Fuel_Type': ['Petrol', 'Diesel', 'Petrol', 'Diesel', 'Diesel', 'Petrol', 'Diesel', 'Petrol'],
9          'Driven_kms': [27000, 43000, 6900, 5200, 42450, 2071, 41000, 42000],
10         'Selling_Price': [3.35, 4.75, 7.25, 2.85, 4.60, 9.25, 6.75, 6.50] # In thousands/millions
11     }
12     df = pd.DataFrame(data)
13
14
15     df = df.dropna()
16
17     current_year = 2026
18     df['Age'] = current_year - df['Year']
19     df = df.drop(columns=['Year'])
20
21     return df
22
23
24 from sklearn.model_selection import train_test_split
25 from sklearn.ensemble import RandomForestRegressor
26 from sklearn.preprocessing import OneHotEncoder
27 from sklearn.compose import ColumnTransformer
28 from sklearn.pipeline import Pipeline
29
30 def train_model(df):
31     X = df.drop(columns=['Selling_Price'])
32     y = df['Selling_Price']
33
34     categorical_features = ['Fuel_Type']
35     numerical_features = ['Driven_kms', 'Age']
36
37     preprocessor = ColumnTransformer(
38         transformers=[
39             ('num', 'passthrough', numerical_features),
40             ('cat', OneHotEncoder(), categorical_features)
41         ])

```

```

    model_pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('regressor', RandomForestRegressor(n_estimators=100, random_state=42))
    ])

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model_pipeline.fit(X_train, y_train)

    return model_pipeline, X, preprocessor

import streamlit as st
import plotly.express as px

st.set_page_config(page_title="Car Price Predictor", layout="wide")
st.title("🚗 Used Car Price Prediction Dashboard")
st.write("Predict the valuation of a used car based on its historical features.")

df = load_and_clean_data()
model, X, preprocessor = train_model(df)

col1, col2 = st.columns([1, 2])

with col1:
    st.header("📋 Car Specifications")

    input_year = st.slider("Manufacturing Year", min_value=2010, max_value=2026, value=2018)
    input_age = 2026 - input_year

    input_mileage = st.number_input("Driven Kilometers (Mileage)", min_value=0, value=30000, step=1000)

    fuel_options = df['Fuel_Type'].unique()
    input_fuel = st.selectbox("Fuel Type", options=fuel_options)

    predict_btn = st.button("Predict Selling Price", type="primary")

```

```

with col2:
    if predict_btn:
        # Construct dataframe for prediction matching training format
        user_data = pd.DataFrame({
            'Fuel_Type': [input_fuel],
            'Driven_kms': [input_mileage],
            'Age': [input_age]
        })

        prediction = model.predict(user_data)[0]

        st.success(f"Predicted Value: ${prediction:.2f} Lakhs")

        st.subheader("📊 Feature Importance Analysis")

        importances = model.named_steps['regressor'].feature_importances_

        cat_encoder = model.named_steps['preprocessor'].named_transformers_['cat']
        encoded_cat_names = cat_encoder.get_feature_names_out(['Fuel_Type']).tolist()
        feature_names = ['Driven_kms', 'Age'] + encoded_cat_names

        importance_df = pd.DataFrame({
            'Feature': feature_names,
            'Importance': importances
        }).sort_values(by='Importance', ascending=True)

        fig_importance = px.bar(
            importance_df,
            x='Importance',
            y='Feature',
            orientation='h',
            title="What factors influence the price most?",
            labels={'Importance': 'Relative Importance Factor'},
            color='Importance',
            color_continuous_scale='Blues'
        )
        st.plotly_chart(fig_importance, use_container_width=True)

        st.subheader("📈 Mileage vs Price Trend")
        fig_scatter = px.scatter(
            df, x='Driven_kms', y='Selling_Price', color='Fuel_Type',
            title="Dataset Distribution: Mileage vs Price",
            labels={'Driven_kms': 'Kilometers Driven', 'Selling_Price': 'Price'}
        )

        fig_scatter.add_scatter(x=[input_mileage], y=[prediction], mode='markers',
                               marker=dict(size=15, color='red', symbol='star'),
                               name='Your Prediction')
        st.plotly_chart(fig_scatter, use_container_width=True)
    else:
        st.info("Adjust the car specifications on the left and click **Predict Selling Price** to generate charts.")

```

Conclusion

- The dashboard predicts car prices using regression modeling.
- Feature importance visualization highlights which factors matter most.
- Streamlit interface allows users to input values and instantly see predictions.
- Charts make the results clear and visually appealing.
- This system can be extended with more features (brand, engine size, transmission type).